

Assignment No 13

Name : Shivam Ramakant Deshmane

Domain : AI & ML

College : T.B Girwalkar Polytechnic Ambajogai

Q1.

Code :

```
import numpy as np
```

```
# a) Create a 1D array
```

```
array1 = np.array([10, 20, 30, 40, 50])
```

```
# b) Create a 2D array (3x3)
```

```
array2 = np.array([
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
])
```

```
print("1D Array:")
```

```
print(array1)
```

```
print("Shape:", array1.shape)
```

```
print("Data Type:", array1.dtype)
```

```
print("\n2D Array:")
```

```
print(array2)
```

```
print("Shape:", array2.shape)
```

```
print("Data Type:", array2.dtype)
```

Output :

```
1D Array:  
[10 20 30 40 50]  
Shape: (5,)  
Data Type: int64
```

```
2D Array:  
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]  
Shape: (3, 3)  
Data Type: int64
```

Q2.

Code :

```
import numpy as np
```

```
# a) 1D array of 8 zeros
```

```
array1 = np.zeros(8)
```

```
# b) 2D array of shape (4, 4) filled with ones
```

```
array2 = np.ones((4, 4))
```

```
# c) 3x3 matrix of zeros
```

```
array3 = np.zeros((3, 3))
```

```
# Print all arrays with proper labels
```

```
print("a) 1D Array of 8 Zeros:")
```

```
print(array1)
```

```
print("\nb) 4x4 Array of Ones:")
```

```
print(array2)
```

```
print("\nc) 3x3 Matrix of Zeros:")
```

```
print(array3)
```

Output :

a) 1D Array of 8 Zeros:

```
[0. 0. 0. 0. 0. 0. 0. 0.]
```

b) 4x4 Array of Ones:

```
[[1. 1. 1. 1.]  
 [1. 1. 1. 1.]  
 [1. 1. 1. 1.]  
 [1. 1. 1. 1.]]
```

c) 3x3 Matrix of Zeros:

```
[[0. 0. 0.]  
 [0. 0. 0.]  
 [0. 0. 0.]]
```

Q3.

Code :

```
import numpy as np
```

```
array1 = np.arange(0, 21, 1)
print("a) Numbers from 0 to 20:")
print(array1)
```

```
array2 = np.arange(10, 51, 2)
print("\nb) Even numbers from 10 to 50:")
print(array2)
```

```
array3 = np.arange(5, 101, 5)
print("\nc) Numbers from 5 to 100 with step of 5:")
print(array3)
```

Output :

a) Numbers from 0 to 20:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14
```

b) Even numbers from 10 to 50:

```
[10 12 14 16 18 20 22 24 26 28 30 32 34 36 38
```

c) Numbers from 5 to 100 with step of 5:

```
[ 5 10 15 20 25 30 35 40 45 50 55  
 95 100]
```

Q4.

Code :

```
import numpy as np
```

```
arr1 = np.linspace(0, 5, 10)
```

```
arr2 = np.linspace(-10, 10, 15)
```

```
print("a) 10 equally spaced numbers between 0 and 5:")
```

```
print(arr1)
```

```
print("Length:", len(arr1))
```

```
print("\nb) 15 equally spaced numbers between -10 and 10:")
```

```
print(arr2)
```

```
print("Length:", len(arr2))
```

Output :

```
a) 10 equally spaced numbers between 0 and 5:  
[0.          0.55555556  1.11111111  1.66666667  2.  
 3.33333333  3.88888889  4.44444444  5.          ]  
Length: 10
```

```
b) 15 equally spaced numbers between -10 and 1  
[-10.         -8.57142857 -7.14285714 -5.71428571  
 -2.85714286 -1.42857143  0.          1.42857143  
  4.28571429  5.71428571  7.14285714  8.57142857  
Length: 15
```


Q5.

Code :

```
import numpy as np
array1 = np.random.rand(10)
matrix = np.random.randn(3, 3)
print("a) 1D Array of 10 random numbers between 0 and 1:")
print(array1)
print("\nb) 3x3 Matrix of random numbers from standard normal distribution:")
print(matrix)
```

Output :

```
a) 1D Array of 10 random numbers between 0 and
[0.72793255 0.18629201 0.767293    0.74698346 0
 0.75397376 0.98980547 0.37493705 0.29455559]

b) 3x3 Matrix of random numbers from standard
[[ 1.26357866  1.55271552 -1.43128386]
 [ 2.27112549 -1.35259098 -0.64384836]
 [-1.99308279  0.20496535 -0.06436438]]
```

Q6.

Code :

```
import numpy as np

# Create two vectors
v1 = np.array([2, 4, 6, 8])
v2 = np.array([1, 3, 5, 7])

# Perform operations
addition = v1 + v2
subtraction = v1 - v2
multiplication = v1 * v2
dot_product = np.dot(v1, v2)

# Print results
print("Vector 1:", v1)
print("Vector 2:", v2)

print("\nAddition:", addition)
print("Subtraction:", subtraction)
print("Element-wise Multiplication:", multiplication)
print("Dot Product:", dot_product)
```

Output :

```
Vector 1: [2 4 6 8]
```

```
Vector 2: [1 3 5 7]
```

```
Addition: [ 3  7 11 15]
```

```
Subtraction: [1 1 1 1]
```

```
Element-wise Multiplication: [ 2 12 30 56]
```

```
Dot Product: 100
```

Q7.

Code :

```
import numpy as np
```

```
A = np.array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])
```

```
B = np.array([[9, 8, 7],  
              [6, 5, 4],  
              [3, 2, 1]])
```

```
# Matrix addition
```

```
addition = A + B
```

```
matrix_multiplication = A @ B    # or np.dot(A, B)
```

```
elementwise_multiplication = A * B
```

```
print("Matrix A:")
```

```
print(A)
```

```
print("\nMatrix B:")
```

```
print(B)
```

```
print("\nMatrix Addition (A + B):")
```

```
print(addition)
```

```
print("\nMatrix Multiplication (A @ B):")
```

```
print(matrix_multiplication)
```

```
print("\nElement-wise Multiplication (A * B):")
```

```
print(elementwise_multiplication)
```

Matrix A:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Matrix B:

```
[[9 8 7]
 [6 5 4]
 [3 2 1]]
```

Matrix Addition (A + B):

```
[[10 10 10]
 [10 10 10]
 [10 10 10]]
```

Matrix Multiplication (A @ B):

```
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

Element-wise Multiplication (A * B):

```
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]
```

Q8.

Code :

```
import numpy as np
```

```
arr = np.random.randint(1, 51, 20)
```

```
matrix = arr.reshape(4, 5)
```

```
# Print the matrix
```

```
print("4x5 Matrix:")
```

```
print(matrix)
```

```
# Print sum, mean, and standard deviation
```

```
print("\nSum of all elements:", np.sum(matrix))
```

```
print("Mean of all elements:", np.mean(matrix))
```

```
print("Standard Deviation:", np.std(matrix))
```

```
# Find the maximum value in each row
```

```
print("\nMaximum value in each row:")
```

```
print(np.max(matrix, axis=1))
```

Output :

```
4x5 Matrix:
```

```
[[19 26  5 38 36]
 [24 45 16 24 21]
 [34 17  5 26 18]
 [49 49 37 22 15]]
```

```
Sum of all elements: 526
```

```
Mean of all elements: 26.3
```

```
Standard Deviation: 12.63368513142543
```

```
Maximum value in each row:
```

```
[38 45 34 49]
```


Q9.

Code :

```
import numpy as np

arr = np.random.randint(1, 51, 20)

print("1D Array:")
print(arr)

matrix = arr.reshape(4, 5)

print("\n4x5 Matrix:")
print(matrix)

# Calculate statistics
print("\nSum of all elements:", np.sum(matrix))
print("Mean of all elements:", np.mean(matrix))
print("Standard Deviation:", np.std(matrix))

# Find maximum value in each row
print("\nMaximum value in each row:")
print(np.max(matrix, axis=1))
```

Output :

```
1D Array:
```

```
[38 11  2 29 34 34 29 19 12  8 15 35 40 43 48]
```

```
4x5 Matrix:
```

```
[[38 11  2 29 34]
 [34 29 19 12  8]
 [15 35 40 43 48]
 [46 47  4  7  4]]
```

```
Sum of all elements: 505
```

```
Mean of all elements: 25.25
```

```
Standard Deviation: 15.795173313389126
```

```
Maximum value in each row:
```

```
[38 34 48 47]
```

Q10.

Code :

```
import numpy as np

try:

    # Ask the user how many numbers to generate
    n = int(input("Enter how many numbers you want to generate: "))

    if n <= 0:

        print("Please enter a positive number.")

    else:

        # Generate random numbers between 10 and 100
        arr = np.random.randint(10, 101, size=n)

        # Display the array
        print("\nGenerated Array:")
        print(arr)

        # Display statistics
        print("\nStatistics:")
        print("Mean =", np.mean(arr))
        print("Median =", np.median(arr))
        print("Standard Deviation =", np.std(arr))
```

```

print("Minimum =", np.min(arr))
print("Maximum =", np.max(arr))

# Reshape into a 2D array if possible
rows = int(np.sqrt(n))

while rows > 1:
    if n % rows == 0:
        cols = n // rows
        matrix = arr.reshape(rows, cols)

        print("\nReshaped 2D Array:")
        print(matrix)

        print("\nRow-wise Sum:")
        print(np.sum(matrix, axis=1))
        break

    rows -= 1
else:
    print("\nCannot reshape into a proper 2D array.")

except ValueError:
    print("Error: Please enter a valid integer.")

except Exception as e:
    print("An unexpected error occurred:", e)

```

Enter how many numbers you want to generate: 5

Generated Array:

```
[ 73  17  32  51  67  56  35  24  21  97  49
 30  76  85  91  26  94  47  76  39  31  64
 31  33  49  23  21 100  25  93  80  31  80
 68]
```

Statistics:

Mean = 52.78181818181818

Median = 51.0

Standard Deviation = 24.925847051598257

Minimum = 14

Maximum = 100

Reshaped 2D Array:

```
[[ 73  17  32  51  67  56  35  24  21  97  49]
 [ 58  16  48  75  43  83  33  30  76  85  91]
 [ 26  94  47  76  39  31  64  72  55  39  76]
 [ 75  21  71  31  33  49  23  21 100  25  93]
 [ 80  31  80  32  17  81  14  76  51  52  68]]
```

Row-wise Sum:

```
[522 638 619 542 582]
```

